

TECHNICAL REFERENCE

Wine DAW Compatibility Patches

FL Studio Authenticode validation and hosted OpenGL/
VST presentation ordering

WineDAWPatches project

6 September 2026

`crypt32 authenticated-attribute compatibility • win32u client-surface ordering`

CONTENTS

Purpose and scope

- Patch inventory
- Corrected behaviors

Part I — FL Studio validation compatibility

- Symptom and localization
- Authenticated attributes and signed bytes
- Patch 1: a test that separates compatibility from bypass
- Patch 2: verification-only ordered encoding
- Why the validation fix is narrowly scoped
- Validation evidence

Part II — Hosted OpenGL/VST flicker

- Symptom and controls
- Wine/X11 rendering model in this case
- The observed overwrite
- Rejected hypotheses
- The ordering invariant
- Changes in `dlls/win32u/dce.c`
- Changes in `dlls/win32u/window.c`
- Declaration in `dlls/win32u/win32u_private.h`
- Locking and ordering review
- Scope and cost
- Flicker verification evidence

Build lineage and reproducibility

- Pinned base
- Patch order
- Applying the exported patches
- Rollback

Safety, limitations, and non-goals

- Security properties retained
- Graphics properties retained
- Non-goals

Troubleshooting by observed result

Glossary

Source map

Purpose and scope

This document describes two application-independent Wine compatibility fixes developed while running a digital-audio workstation and plug-ins under Wine:

1. a `crypt32` correction that allows Windows-compatible verification of CMS signatures whose authenticated attributes are not stored in canonical DER order; and
2. a `win32u` presentation-order correction that prevents an overlapping software parent surface from leaving a nested, offscreen OpenGL or Vulkan client surface covered.

The first problem appeared as the FL Studio 26.1.5.5618 message “**The validity of the program could not be verified.**” The second appeared as a hosted OpenGL/VST3 editor that alternated between its correct image and blank FL Studio wrapper content during window movement and repainting.

User summary. The validation patch makes Wine hash the same authenticated attribute byte sequence that Windows verifies. The flicker patch restores a nested accelerated surface after an overlapping software repaint. Neither patch disables a security check, changes FL Studio registration, or contains a product-specific allowlist.

The report is layered. The first sections explain what changed and what users should expect. The later sections document the byte-level signature failure, X11 event ordering, modified Wine functions, locking rules, performance scope, and verification evidence for reviewers and maintainers.

Patch inventory

Area	Patch	Wine files changed	Purpose
Signature regression	<code>patches/flstudio-authat-trs/0001-crypt32-tests-Test-signatures-with-unsorted-authenti.patch</code>	<code>dlls/crypt32/tests/msg.c</code>	Adds a valid CMS fixture with non-DER-sorted authenticated attributes and a signature-tampered negative case.
Signature implementation	<code>patches/flstudio-authat-trs/0002-crypt32-Preserve-authenticated-attribute-order-when-.patch</code>	<code>dlls/crypt32/msg.c</code>	Preserves decoded authenticated-attribute order only while verifying an existing signature.
Graphics implementation	<code>patches/opengl-flicker/0001-win32u-Re-present-offscreen-client-surfaces-after-wi.patch</code>	<code>dlls/win32u/dce.c</code> , <code>dlls/win32u/win32u_private.h</code> , <code>dlls/win32u/window.c</code>	Re-presents an overlapping offscreen client surface after a successful top-level software-surface flush.

The graphics patch is recorded as Wine commit `3693d0baf73042cf0417ae1856ffcc26332630f0`, titled `win32u: Re-present offscreen client surfaces after window flushes`. Its canonical exported patch lives at the path shown in the table.

Corrected behaviors

Trigger	Before the patch	After the patch
Valid signature with authenticated attributes encoded in a noncanonical order	Wine reconstructs a differently ordered byte stream, rejects the RSA signature, and returns <code>TRUST_E_CERT_SIGNATURE (0x80096004)</code> through <code>WinVerifyTrust</code> .	Wine preserves the decoded order for verification, reconstructs the signed byte stream, and accepts the valid signature.
Same CMS message with a changed signature byte	Rejected.	Still rejected with <code>NTE_BAD_SIGNATURE</code> ; the compatibility fix does not force success.
Nested redirected GL client followed by an overlapping parent software flush	The parent can become the last writer to the shared top-level drawable, leaving the plug-in area blank until a later client presentation.	Wine re-presents matching offscreen client surfaces after the successful parent flush, so the accelerated child becomes the final writer in the overlap.
Unrelated or non-overlapping client surface	Normal presentation path.	Unchanged; the helper filters by top-level window, offscreen state, and rectangle intersection.

Part I — FL Studio validation compatibility

Symptom and localization

FL Studio 26.1.5.5618 aborted startup under the unpatched Wine runner with:

The validity of the program could not be verified.

Launching `FL64.exe` directly reproduced the result. The same Wine prefix had run an older FL Studio version, which made a launcher-only or damaged-prefix explanation unlikely. Wine traces localized the failure to Authenticode verification of `FLEngine_x64.dll`.

The embedded signature used a SHA-256 digest and a 3072-bit RSA signing certificate. Parsing the signed content and certificate chain succeeded. Decrypting the RSA signature yielded a valid PKCS#1 v1.5 SHA-256 `DigestInfo`. The failure occurred because Wine hashed different authenticated-attribute bytes from those covered by the signature.

The externally visible error chain was:

```

FL64.exe startup validation
↓
WinVerifyTrust(FLEngine_x64.dll)
↓
crypt32 CMS/PKCS#7 signature verification
↓
authenticated attributes re-encoded in a different order
↓
CryptVerifySignatureW rejects the mismatched digest
↓
TRUST_E_CERT_SIGNATURE (0x80096004)

```

This distinction matters: the content, certificate decoding, and RSA primitive were not broken. The bytes supplied to the digest calculation were wrong for this already-existing signature.

Authenticated attributes and signed bytes

CMS **SignerInfo** can contain authenticated, also called signed, attributes. Typical examples identify the content type and contain the message digest. If authenticated attributes are present, the signature covers their encoded **SET OF Attribute** value rather than directly covering the document digest.

DER normally imposes a canonical sort order on elements of a **SET OF**. The affected signature, however, had been generated over the attributes in their original encoded order. Windows accepts that signature. Wine decoded the attributes successfully, but the old verification path later passed the decoded collection to its generic **PKCS_ATTRIBUTES** encoder. That encoder DER-sorted the set, creating a valid encoding with a different byte sequence.

```

Bytes in the signed CMS message
  Attribute A, Attribute B, Attribute C, Attribute D
      | decode
      ▼
Wine CRYPT_ATTRIBUTES array
  [A, B, C, D]
      | old generic PKCS_ATTRIBUTES encode
      ▼
Canonical DER order
  [C, A, D, B]
      ← illustrative order; bytes differ
      | hash and verify
      ▼
Digest does not match the digest protected by the RSA signature

```

The illustrative letters above explain the transformation; they are not the actual attribute names or their exact sort order. The important invariant is that generic **SET OF** encoding may reorder members, while the compatible verification behavior preserves the sequence obtained from the signed input.

For the diagnosed Image-Line binary, the two SHA-256 values were:

Byte stream	SHA-256
Digest expected by the signature	35f9c2795cedc7c124bb93f4710f2eb264ae20bbf7258093f4b52fd-cf8520ef6
Digest of Wine's old reconstruction	0fabe424b0a535bc80f7993399ddf244b0d1a4aca79f0e63d7f-b3b7697608201

Because the hashes were different, correct RSA verification had to reject the old reconstruction. The solution was to reconstruct the compatible bytes—not to weaken the cryptographic comparison.

Patch 1: a test that separates compatibility from bypass

The first patch adds `signedWithUnsortedAuthAttrsBareContent` to `dlls/crypt32/tests/msg.c`. The fixture is a compact signed CMS message whose authenticated attributes are deliberately not in DER-sorted order.

The test performs two checks through `CryptMsgControl` with `MSG_CTRL_VERIFY_SIGNATURE`:

1. The unchanged fixture must verify successfully.
2. A copy with the last signature byte toggled using `^= 1` must fail, and `GetLastError()` must be exactly `NTE_BAD_SIGNATURE`.

The pair is essential. A test containing only the valid message could also be satisfied by an accidental or overly broad “accept” path. Requiring the nearly identical tampered message to fail proves that cryptographic verification continues to control the result.

Security invariant. Valid noncanonical ordering is compatible; an invalid signature is not. The negative test guards that boundary explicitly.

Patch 2: verification-only ordered encoding

The implementation patch changes `CSignedMsgData_UpdateAuthenticatedAttributes()` in `dlls/crypt32/msg.c` and adds this internal helper:

```
static BOOL encode_attributes_in_order(const CRYPT_ATTRIBUTES *attributes,
    BYTE **encoded, DWORD *size);
```

What `encode_attributes_in_order()` builds

The helper reconstructs a complete ASN.1 `SET OF` while retaining the decoded array order:

1. It calls `CryptEncodeObjectEx(..., PKCS_ATTRIBUTE, ...)` once per attribute to obtain its encoded size.
2. It adds those sizes with overflow checks. A value that would exceed the representable ASN.1 size fails with `CRYPT_E_ASN1_LARGE`.
3. It computes the encoded length-field size with `CRYPT_EncodeLen()`.
4. It allocates one result buffer with `LocalAlloc()`.
5. It writes the constructed `SET OF` tag, `ASN_CONSTRUCTOR | ASN_SETOF`, and the combined payload length.
6. It encodes each `PKCS_ATTRIBUTE` into that buffer in `rgAttr[0]` through `rgAttr[cAttr - 1]` order.
7. If an individual encoding fails, it frees the buffer, clears the output pointer, and propagates failure.

Each attribute is still encoded normally; only the generic whole-collection operation that would sort the members is avoided.

The verification/signing split

The caller already distinguishes signing from verification with the internal `SignOrVerify` enum. The patch uses the new helper only when `flag == Verify`:

```

if (flag == Verify)
    ret = encode_attributes_in_order(&signer_info->AuthAttrs,
                                    &encodedAttrs, &size);
else
    ret = CryptEncodeObjectEx(X509_ASN_ENCODING, PKCS_ATTRIBUTES,
                             &signer_info->AuthAttrs,
                             CRYPT_ENCODE_ALLOC_FLAG, NULL,
                             &encodedAttrs, &size);

```

The pseudocode shortens the actual member expression for readability, but the branch and APIs match the patch.

This split preserves two separate requirements:

- **Verification** must reproduce the byte sequence represented by an existing Windows-compatible signature, including its decoded attribute order.
- **Signing** should keep using the canonical `PKCS_ATTRIBUTES` encoder so Wine emits DER-sorted sets for new signatures.

The existing `CryptHashData()` operation, RSA verification, return handling, and `LocalFree()` lifetime remain in place. The patch does not modify `WinVerifyTrust` policy, certificate-chain evaluation, or the RSA algorithm.

Why the validation fix is narrowly scoped

The change contains no check for Image-Line, FL Studio, a filename, certificate, publisher, path, version, or file hash. It applies whenever the same CMS encoding condition reaches Wine's authenticated-attribute verification path.

It also does **not**:

- return success before cryptographic verification;
- ignore `NTE_BAD_SIGNATURE` or `TRUST_E_CERT_SIGNATURE`;
- alter file content hashing;
- install a certificate or change a trust store;
- remove FL Studio licensing or registration checks; or
- treat trial mode as a verification failure.

FL Studio's trial/registration state is separate. During runtime validation, the patched build displayed normal trial state while the startup validity error was absent. That was expected and intentional.

Validation evidence

Test-first `crypt32` result

With the new regression fixture but without the implementation fix, the Wine 11.16 `msg` test ran 1,030 assertions. Exactly the valid unsorted-attribute assertion failed with `NTE_BAD_SIGNATURE`; the tampered case passed.

After the implementation patch, the same 1,030 assertions completed with zero failures. The valid fixture passed and its signature-tampered copy remained rejected.

Adjacent Wine test coverage

Test area	Assertions	Failures recorded
<code>crypt32</code> message test	1,030	0 after patch
<code>rsaenh</code>	4,099	0
WinTrust ASN	211	0
WinTrust crypt/catalog	378	0
WinTrust registration	40	0
WinTrust <code>softpub</code>	254	0

Fresh `msg.ok`, `rsaenh.ok`, `asn.ok`, `crypt.ok`, `register.ok`, and `softpub.ok` targets also exited successfully against the complete patched Wine-Staging tree.

Real-binary positive and negative matrix

The same pinned `FLEngine_x64.dll` was checked in all rows:

Runner and input	<code>WinVerifyTrust</code> result	Interpretation
System Wine, genuine DLL	<code>80096004</code>	Original compatibility failure reproduced.
Patched private Wine, genuine DLL	<code>00000000</code>	Valid signature accepted.
Patched private Wine, temporary byte-tampered copy	<code>80096010</code>	Invalid signature rejected.

The genuine DLL's recorded SHA-256 was `b85d5274d643a9aad24f898e17de5b5ddf668487e9e0c6968b0e47f5a619b-dd3`. The verification tooling required that exact input, made the negative copy only temporarily, and removed the proprietary copy on exit.

Application result

With the patched runner, `FL64.exe`, its embedded WebView processes, the main FL Studio 2026 window, settings, and welcome window all appeared. The validity dialog did not. This application observation complements rather than replaces the deterministic positive and negative cryptographic tests.

Part II — Hosted OpenGL/VST flicker

Symptom and controls

The tested OpenGL plug-in rendered correctly as a standalone application under the Wine runner. The same renderer hosted as a VST3 editor in FL Studio 26.1.5.5618 could alternate between a complete plug-in image and blank wrapper content, particularly while the wrapper title bar was moved.

Several details ruled out a simple plug-in render failure:

- notes and controls remained responsive while the editor looked blank;
- Wine traces recorded uninterrupted buffer swaps for the same client window and OpenGL context;
- no X11 unmap, destroy, or reparent event preceded the blank frames; and
- detaching or bridging the editor did not prevent the problem.

User summary. The OpenGL plug-in continued drawing. The image disappeared because FL Studio's later software repaint covered the already-presented plug-in pixels in the shared native window. The patch corrects which surface is presented last.

Although a hosted OpenGL plug-in and FL Studio exposed the bug, the affected mechanism is generic to nested Wine client surfaces. There are no vendor, product, VST, executable, window-class, or path checks in the patch.

Wine/X11 rendering model in this case

The hosted editor was a nested child inside FL Studio's single native top-level X11 window. Wine represented its accelerated content with a redirected, offscreen `client_surface`. Presentation copied that surface into the top-level drawable. FL Studio's surrounding interface used a software `window_surface`, whose dirty regions were uploaded to the same drawable.

```
Hosted OpenGL child
  renders and swaps
    ↓
XComposite redirected offscreen client_surface
  ↓ X11 CopyArea
FL Studio shared top-level drawable
  ↑ MIT-SHM PutImage
FL Studio software window_surface flush
```

Both paths were individually working. The bug was their final ordering when the rectangles overlapped. The parent window did not have `WS_CLIPCHILDREN`, so its software dirty rectangle could include the child region.

The observed overwrite

The first captured title-drag marker was `2026-09-02T11:31:44.822192+00:00`. The decoded X11 protocol then showed:

UTC time	X11 operation	Destination and size	Meaning
<code>11:31:44.879067</code>	client <code>CopyArea</code>	<code>(351,330) , 1280×688</code>	Correct redirected OpenGL client copied into the FL top-level drawable.
<code>11:31:44.879954</code>	parent <code>MIT-SHM PutImage</code>	<code>(344,304) , 1296×714</code>	FL software surface uploaded 887 microseconds later; its rectangle completely contained the client image.
<code>11:31:44.898576</code>	next client <code>CopyArea</code>	client region	The plug-in became visible again.

UTC time	X11 operation	Destination and size	Meaning
11:31:44.902806	next parent <code>PutImage</code>	containing parent region	Parent covered it again.

The alternating X11 writes matched the alternating visible frames. Three independent evidence streams agreed:

1. compositor recording captured the visible complete/blank alternation during the exact title-drag sequence;
2. Wine debug output showed the wrapper moving and the same OpenGL client continuing `win32u_wglSwapBuffers` and `x11drv_egl_surface_swap`; and
3. decoded X11 traffic showed a correct client copy followed by an overlapping parent upload to the final drawable.

The causal sequence was therefore:

```

wrapper move or repaint
↓
nested client geometry updated; GL swaps continue
↓
offscreen client surface copied into top-level drawable
↓
overlapping parent software surface flushed afterward
↓
blank wrapper pixels become the last visible write

```

Rejected hypotheses

Hypothesis	Observation	Conclusion
The plug-in cannot render under Wine	Its standalone OpenGL renderer worked correctly.	Rejected.
The plug-in stops drawing while blank	Buffer swaps continued for the same HWND and context.	Rejected.
The child is hidden, destroyed, or re-parented	No corresponding X11 lifecycle event preceded blank frames.	Rejected.
Input freezes with the image	Controls and notes remained usable.	Rejected.
Detached mode avoids the problem	Flicker was reproduced while detached.	Rejected as a workaround.
FL Studio bridged mode avoids the problem	Flicker was reproduced while bridged.	Rejected as a workaround.

These controls are important because the implemented change belongs in Wine's presentation layer. It does not add timing delays to FL Studio, interfere with the plug-in's GL calls, or fabricate paint messages.

The ordering invariant

The patch implements this rule:

After a top-level software window surface is successfully flushed, every offscreen client surface belonging to that top-level and intersecting the flushed rectangle must be re-presented before the operation is complete.

This makes the child the final writer only where it needs to be. It does not prevent the parent flush or require the parent application to use `WS_CLIPCHILDREN`.

Changes in `dlls/win32u/dce.c`

`window_surface_flush_internal()`

The existing `window_surface_flush()` body becomes an internal helper:

```
static BOOL window_surface_flush_internal(struct window_surface *surface,
                                         RECT *flushed_rect);
```

The helper retains the original lock, dirty-region processing, and `surface->funcs->flush()` call. It adds a `flushed` result initialized to `FALSE`. Only when the surface-specific flush succeeds does it:

- reset the accumulated dirty bounds, as before;
- set `flushed = TRUE`; and
- if requested, copy the dirty rectangle and translate it with `OffsetRect(flushed_rect, surface->rect.left, surface->rect.top)`.

That translation puts the dirty rectangle in the coordinate space used by a client surface's `virtual_rect`, allowing a meaningful overlap test later. The helper unlocks the surface and returns whether a real flush succeeded.

The public function then becomes the ordering hook:

```
void window_surface_flush(struct window_surface *surface)
{
    RECT flushed_rect;

    if (window_surface_flush_internal(surface, &flushed_rect))
        present_client_surfaces(surface->hwnd, &flushed_rect);
}
```

Because the call happens after `window_surface_flush_internal()` returns, the individual window-surface mutex is no longer held when client surfaces are enumerated and presented.

Scaled-surface recursion avoidance

`scaled_surface_flush()` already forwards a flush to its target surface. If it called the new public wrapper, that nested operation could start client re-presentation while the outer scaled flush was still in progress.

The patch changes that one call to:

```
window_surface_flush_internal(surface->target_surface, NULL);
```

The inner flush therefore performs the original work without triggering the post-flush presentation hook. Only the outer public operation establishes the final ordering. Passing `NULL` also avoids computing a rectangle that no caller will consume.

Changes in `dlls/win32u/window.c`

The new exported-internally helper is:

```
void present_client_surfaces(HWND hwnd, const RECT *rect);
```

It holds `surfaces_lock` while iterating the global client-surface list. A surface is eligible only if all of these checks pass:

```
surface->toplevel == hwnd
surface->offscreen is true
surface->virtual_rect intersects the flushed rectangle
NtUserGetDCEx() returns an HDC for the client window
```

The offscreen state is read with `InterlockedCompareExchange(..., 0, 0)`, matching its interlocked access model. Intersection is computed with `intersect_rect()`.

For each match, the helper obtains a cached/style-aware device context using `NtUserGetDCEx(surface->hwnd, 0, DCX_CACHE | DCX_USESTYLE)`, calls the surface's own `funcs->present(surface, hdc)`, and releases the DC with `NtUserReleaseDC()`.

The code deliberately calls `surface->funcs->present()` directly instead of `client_surface_present()`. The public client helper also acquires `surfaces_lock`; using it inside the enumeration would recursively acquire the same lock. The direct function-table call performs the presentation without that lock recursion.

Declaration in `dlls/win32u/win32u_private.h`

`win32u_private.h` adds one internal declaration next to the other client-surface functions:

```
extern void present_client_surfaces(HWND hwnd, const RECT *rect);
```

No public Wine API or ABI is added. This declaration only connects the DCE surface-flush code with the internal client-surface implementation.

Locking and ordering review

The relevant lock sequence after the patch is:

```

window_surface_flush_internal()
    lock surface->mutex
    perform software flush
    capture translated dirty rectangle
    unlock surface->mutex
    ↓
present_client_surfaces()
    lock surfaces_lock
    present filtered clients directly
    unlock surfaces_lock

```

The surface mutex is released before `surfaces_lock` is acquired, avoiding a new nested lock order between those two locks. The scaled-surface internal call prevents a post-presentation callback from occurring during recursive target flush processing.

Scope and cost

The post-flush scan considers only client surfaces registered with Wine. It then filters to the same top-level window, requires `offscreen`, and requires a dirty-rectangle intersection. Attached client surfaces, clients belonging to other top-level windows, and non-overlapping clients are not re-presented.

The expected cost is one additional client-surface blit per matching surface after an overlapping successful software flush. That cost is paid precisely where the alternative can leave accelerated child pixels covered. A failed or empty software flush returns false and does not trigger the scan.

The patch does not change OpenGL command submission, swap-interval policy, plug-in scheduling, or Vulkan rendering. It repairs composition ordering after the accelerated image has already been produced.

Flicker verification evidence

Generic ordering regression

`scripts/test-client-surface-ordering.sh` uses a product-independent nested client-surface reproducer and Wine trace ordering as its oracle.

Runner	Final relevant trace event	Result
Unpatched Wine 11.16	Parent software surface flush	Fails: parent can remain above the redirected child.
Patched Wine	Redirected child client presentation	Passes: <code>PASS: redirected child client surface follows the parent software flush.</code>

An in-tree final-screen-pixel assertion was not used because Wine's rootless X11 model does not guarantee that a Win32 screen DC exposes the pixels finally composited by XWayland. The trace regression checks the generic ordering directly; compositor frames provide the end-to-end visual gate.

Focused Wine and combined-stack checks

The focused 64-bit `opengl32` and `win32u` test modules passed with the patch. A complete 32/64-bit runner was built, and the local FL Authenticode verifier remained green: the genuine DLL returned `00000000`, while the temporary tampered input returned `80096010`.

This combined check matters because the graphics patch is applied on top of the authentication-patched tree. It confirms that the resulting runner retains both behaviors.

End-to-end visual stress test

The `fix03` capture contained 406 compositor frames. The failure-triggering movement was repeated with ten back-and-forth wrapper title drags. Every one of the inspected frames 208 through 247 retained the rendered OpenGL editor. Before the patch, equivalent movement produced the deterministic complete/blank alternation documented in the protocol trace.

Build lineage and reproducibility

Pinned base

Component	Revision or version
Upstream Wine	11.16, commit <code>8da89f8493b21ebfbe344a54dbef0cde23c7ea59</code>
Wine-Staging	11.16, commit <code>f1936f02ba06b5012f89f75901944e8c2d26951c</code>
FL Studio used for validation	26.1.5.5618
OpenGL VST3 plug-in used for visual reproduction	Version 7.0.1

The two stable content identifiers recorded for the authentication series are:

```
c226b69a05cca8e40b1fd507ff6de82d4a575050
92e83747f05a6167e63fab306c53ff72745718b8
```

The authentication changes were originally developed as commits `a462bb3` (regression) and `6c7ba16` (implementation). In the prepared Wine lineage they appear as `d2bf5695cab75f759e64934500bb59a906e5eb9a` and `de5d97de0c7d47db142f4249bbac85f319b43862`. Commit IDs can change when a patch is rebased or integrated; the exported patch content and stable patch IDs are the appropriate identity checks.

The graphics implementation is commit `3693d0baf73042cf0417ae1856ffcc26332630f0`. The complete installed runner used for the recorded visual verification reported:

```
wine-11.16-283-g3693d0b (Staging)
```

Patch order

For the tested combined tree, the conceptual order is:

Wine 11.16

- + matching Wine-Staging 11.16 series
- + crypt32 unsorted-authenticated-attribute regression
- + crypt32 verification-order implementation
- + win32u offscreen client-surface ordering implementation

The graphics mail patch was checked with `git apply --check` against both the authentication-patched Wine 11.16 tree and pristine Wine 11.16. The authentication series was also trial-applied without conflicts or whitespace errors to the then-current upstream Wine revision. Those application checks are portability evidence, not a substitute for building and testing a new Wine revision.

Applying the exported patches

For review, apply each mail patch to a clean, compatible Wine source tree with `git am --3way`, or apply its diff with `git apply` where appropriate. The repository scripts pin the tested inputs and record provenance; they are safer than manually assembling an unrecorded tree.

Before building, confirm:

- the upstream Wine and Wine-Staging revisions match;
- both authentication patches are present and ordered test-before-fix;
- the graphics patch is applied after the base and staging changes;
- `git diff --check` or `git apply --check` reports no whitespace/context problems; and
- both 32-bit and 64-bit components are built when the target plug-ins require both architectures.

Use the desired build parallelism explicitly, for example `make -j "$(nproc)"`, only if the machine and build configuration are stable under full concurrency. The test results in this report describe the recorded build, not every possible compiler, staging series, or later Wine revision.

Rollback

For a source tree where a patch has not been committed, reverse the exact exported patch with `git apply -R` after first checking that the tree contains no overlapping edits. For a versioned private installation, the safest rollback is to point the runner symlink or launcher at the previous known-good Wine installation.

Do not delete the Wine prefix as part of patch rollback. The prefix contains application files, configuration, and registration state; none of these patches requires recreating it. Likewise, installing or removing a distro Wine package is separate from switching a launcher to a private Wine runner.

Safety, limitations, and non-goals

Security properties retained

- CMS content and attributes are still hashed.
- RSA signature verification still determines acceptance.
- A changed signature byte remains rejected.
- The signing path still emits canonical attribute encoding.
- Certificate and `WinVerifyTrust` policy code is not bypassed.
- No product, publisher, hash, or filename is automatically trusted.

Graphics properties retained

- The parent software flush still occurs.
- Accelerated clients keep their normal presentation implementation.
- Re-presentation is limited to overlapping offscreen clients of the same top-level window.
- Locking is sequenced so the window-surface mutex is released first.
- Scaled nested flushes avoid recursive post-presentation.

Non-goals

These patches do not install FL Studio, third-party plug-ins, PipeASIO, WineASIO, winetricks, audio services, graphics drivers, certificates, or Microsoft runtimes. They do not configure a Wine prefix, ASIO device, DAW buffer size, or desktop launcher. They also do not alter FL Studio licensing, unlock trial features, or guarantee compatibility with arbitrary Wine revisions.

If audio fails while both documented checks pass, diagnose that as a separate audio/ASIO configuration problem. If a plug-in stops swapping or produces an incorrect framebuffer rather than being overwritten, diagnose that as a different renderer or host interaction.

Troubleshooting by observed result

Observation	Most likely boundary to inspect
<code>WinVerifyTrust</code> still returns <code>80096004</code> for the pinned genuine DLL	Confirm the launcher is using the patched runner and that both <code>crypt32</code> patches are in its exact source provenance.
Genuine input passes but the tampered copy also passes	Stop using the build; the negative security invariant has failed. Re-run the focused <code>crypt32</code> and local tamper tests.
Plug-in flickers and traces show parent flush after child presentation	Confirm the <code>win32u</code> ordering patch is present in the runner actually launching FL Studio.
Plug-in flickers but GL swaps stop	The documented overwrite mechanism is not established; inspect host scheduling or GL context lifecycle.
Plug-in remains blank and client frames are already blank	Inspect framebuffer/context state rather than post-presentation ordering.
Standalone and hosted rendering both fail	Check driver, Vulkan/OpenGL translation, architecture, and plug-in dependencies before attributing the result to hosted composition.

Glossary

ASN.1

A data-description notation used by certificates and CMS. Its binary encoding rules determine the exact bytes that are hashed and signed.

Authenticated attributes / signed attributes

CMS metadata included in the signature calculation, commonly including the content type and message digest.

Authenticode

Microsoft's code-signing format and verification conventions for Windows binaries, layered on CMS/PKCS#7 and exposed through APIs such as `WinVerifyTrust`.

CMS / PKCS#7

Cryptographic message formats used to carry signed content, certificates, signer information, and attributes.

DER

Distinguished Encoding Rules, a canonical ASN.1 encoding. For `SET OF`, DER normally sorts encoded members.

Client surface

Wine's representation of accelerated child-window content that may be redirected offscreen and then presented into a top-level drawable.

Window surface

Wine's software backing surface for window content and dirty-region flushes.

XComposite redirection

An X11 mechanism that renders a window into an offscreen pixmap so its pixels can later be composed elsewhere.

CopyArea

The X11 operation observed copying the redirected accelerated client into the shared top-level drawable.

MIT-SHM PutImage

The shared-memory X11 image upload observed flushing FL Studio's software parent surface into the same drawable.

Source map

Subject	Repository source
Authentication regression	<code>patches/flstudio-authattrs/0001-crypt32-tests-Test-signatures-with-unsorted-authenti.patch</code>
Authentication fix	<code>patches/flstudio-authattrs/0002-crypt32-Preserve-authenticated-attribute-order-when-.patch</code>
Authentication verification record	<code>docs/verification/2026-08-31-wine-11.16-fl-studio.md</code>
Graphics patch export	<code>patches/opengl-flicker/0001-win32u-Re-present-offscreen-client-surfaces-after-wi.patch</code>
Graphics root-cause record	<code>docs/verification/2026-09-02-opengl-flicker-root-cause.md</code>
Generic graphics regression	<code>scripts/test-client-surface-ordering.sh</code>

This document summarizes the patch artifacts and local verification records. It does not redistribute application or plug-in binaries.